

Assignment 5 SQAT — Automated Tests (Python)

Student: _____

Group: _____

Date: January 2026

Goal

Implement automated UI tests with: (1) test lifecycle annotations, (2) logging, (3) report + screenshots on failure.

Tested web application

<https://the-internet.herokuapp.com/login> (positive and negative login scenarios).

Tools

Python 3, Selenium WebDriver, pytest, webdriver-manager, Python logging, pytest-html.

1) Test lifecycle annotations (TestNG equivalent)

Pytest fixtures are used as an equivalent to TestNG annotations:

- session fixture (autouse) ≈ @BeforeSuite / @AfterSuite
- class fixture (autouse) ≈ @BeforeClass / @AfterClass
- function fixture (driver) ≈ @BeforeMethod / @AfterMethod

```
import logging
import pytest

from utils.logging_config import configure_logging
from utils.driver_factory import create_driver
from utils.screenshot import save_screenshot

log = logging.getLogger("confTest")

# --- Equivalent of @BeforeSuite / @AfterSuite ---
@pytest.fixture(scope="session", autouse=True)
def session_setup_teardown():
    configure_logging()
    log.info("=== TEST SUITE STARTED (session) ===")
    yield
    log.info("=== TEST SUITE FINISHED (session) ===")

# --- Equivalent of @BeforeClass / @AfterClass ---
@pytest.fixture(scope="class", autouse=True)
def class_setup_teardown(request):
    log.info(f"--- TEST CLASS STARTED: {request.node.name} ---")
    yield
    log.info(f"--- TEST CLASS FINISHED: {request.node.name} ---")

# --- Equivalent of @BeforeMethod / @AfterMethod ---
@pytest.fixture(scope="function")
def driver(request):
    log.info(f"Driver setup for test: {request.node.name}")
    d = create_driver()
    yield d
    log.info(f"Driver teardown for test: {request.node.name}")
    d.quit()

# --- Hook: take screenshot on failure and attach to pytest-html report ---
@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    rep = outcome.get_result()

    if rep.when == "call" and rep.failed:
        drv = item.funcargs.get("driver", None)
        if drv is not None:
            path = save_screenshot(drv, item.name)
            extra = getattr(rep, "extra", [])
            try:
                from pytest_html import extras
                extra.append(extras.image(str(path)))
            except Exception:
                pass
            rep.extra = extra
```

2) Logging (Log4j equivalent)

Logging is configured to output to console and to a log file (reports/test-run.log). Test steps are logged.

```
import logging
from pathlib import Path

def configure_logging() -> None:
    """Configure logging similar to Log4j (console + file)."""
    logs_dir = Path("reports")
    logs_dir.mkdir(parents=True, exist_ok=True)
    log_file = logs_dir / "test-run.log"

    root = logging.getLogger()
    root.setLevel(logging.INFO)

    # Avoid duplicate handlers if called multiple times
    if root.handlers:
        return

    fmt = logging.Formatter("%(asctime)s [%(levelname)s] %(name)s - %(message)s")

    console = logging.StreamHandler()
    console.setLevel(logging.INFO)
    console.setFormatter(fmt)

    fileh = logging.FileHandler(log_file, encoding="utf-8")
    fileh.setLevel(logging.INFO)
    fileh.setFormatter(fmt)

    root.addHandler(console)
    root.addHandler(fileh)
```

Log file: **reports/test-run.log**

3) Report + Screenshots (Extent equivalent)

pytest-html generates an HTML report. A pytest hook captures a screenshot on failure and embeds it into the report.

pytest generates HTML report: reports/pytest-report.html (pytest-html). Screenshots on failure are via pytest hook.

HTML report: **reports/pytest-report.html**

Example screenshot of the generated report:

```
PYTEST HTML REPORT (example)
Suite: Assignment 5 SQAT (Python)
Tests: 2
Passed: 2
Failed: 0
```

If a test fails:

- Screenshot is saved to reports/screenshots/
- Screenshot is embedded into the HTML report under the failed test

How to run on macOS M1

1) Create venv and install dependencies:

```
python3 -m venv .venv source .venv/bin/activate pip install -r requirements.txt
```

2) Run tests:

```
pytest
```

Headless:

```
HEADLESS=1 pytest
```

Outputs: HTML report, logs, and screenshots are saved in the **reports/** folder.